



What Language Should You Learn Next?

Prior to learning a new programming language, try and answer a few important questions. Thoughtful introspection will ensure that you get the most out of what you learn – both personally and professionally.

Earlier, it was pretty easy to decide what language to learn next. For example, during my college days in the 90s, the choice was simple – learn C and C++ (though they used to teach Fortran, Cobol, Basic and Pascal in my college as part of the curriculum). Then it was Java. On my own, I dabbled with languages like Smalltalk, Objective-C, Groovy, C#, Go, Prolog, Modula, OCaml and Dylan. It was fun playing around with these languages. But I also wanted to know about the real-world experiences in learning and using different languages. So, I talked to some of my friends who work in the software industry and asked them to share their experiences in learning and using new languages for their regular work.

A friend, who is involved in big data, analytics and cloud computing, is learning and using Scala for his work. Scala is a language that targets the Java platform; it merges both object-oriented and functional features. Because of its functional capabilities, Scala is good for writing concurrent code and for analysing large quantities of data. He recommended learning the language if I worked in this field, but also warned about the unreadable code one can end up with when writing in Scala.

Another friend works on developing software for the financial domain and is currently learning Groovy, which has the productivity of scripting languages in Java-like syntax. In fact, Groovy is more or less a superset of Java. Since he is a Java programmer and wanted to become more productive, he started learning Groovy, which also has meta-programming capabilities and, hence, is one of the popular choices to write Domain Specific Languages (DSLs). Though he is happy about becoming more productive with Groovy, he is also frustrated about having to spend quite a bit of time debugging Groovy code (which took away some of his productivity gains).

Another old friend of mine had an unusual transition from fixing bugs in a C/C++ compiler to programming the Web using JavaScript! She is exploring a growing list of languages like Dart and TypeScript that are trying to replace JavaScript. However, she finds CoffeeScript to be a sweet-spot, since CoffeeScript compiles to JavaScript and she can seamlessly use her existing JavaScript code and third-party libraries in it.

Suggestions for learning new languages based on real-

world experience include: Clojure (with a warning not to give up learning because of its Lisp syntax), Ruby and Python (for a wide range of programming tasks—from automation to Web programming), and Scala (for concurrent programming).

To try out new languages, I recently bought a book titled ‘Seven Languages in Seven Weeks’ (Bruce A Tate, The Pragmatic Bookshelf, 2010). It is an excellent book that gives you a quick overview of the following seven languages: Ruby, Io, Prolog, Scala, Erlang, Clojure and Haskell. I highly recommend this book to anyone who enjoys programming and wants to learn a new language. I liked Ruby in this list, but when I checked with my friend who read the same book, he said he liked Scala and Clojure. Thinking about what language one should learn next, I realised that this is a deeply personal matter. Just like preferences for different cuisines and brands differ from person-to-person, choosing a programming language depends on an individual’s personal likes and dislikes, preferences and the kind of work one does. Hence, before deciding on which language to learn next, seriously consider the following factors.

How much time can you afford to spend on learning a new language?

Learning a new language in an entirely new paradigm will require more effort and time from you. For example, if you are a seasoned Java developer and write Java code on a day-to-day basis, a good choice will be to learn Groovy, since you can seamlessly switch back and forth between Java and Groovy. So you don’t need to dedicate time exclusively to learning Groovy.

Does the new language help you solve your problems more effectively?

A friend of mine, who was a Java developer, wanted to develop an engine for implementing business rules. Hearing about the use of Prolog in rule-based systems, he started learning it. He fell in love with the unusual way that he could think and solve problems in this logical programming language, which is best suited for solving his problems and was happy that he tried it. When choosing your next language, think about the problems you need to solve, and see if it is in sync with the underlying problem-solving approach of your new programming language.